

RNN (Recurrent Neural Network)

An **RNN (Recurrent Neural Network)** is a type of neural network that is designed to **handle sequential data** — like time series, text, audio, or video — by maintaining a **memory of previous inputs** through its internal state.

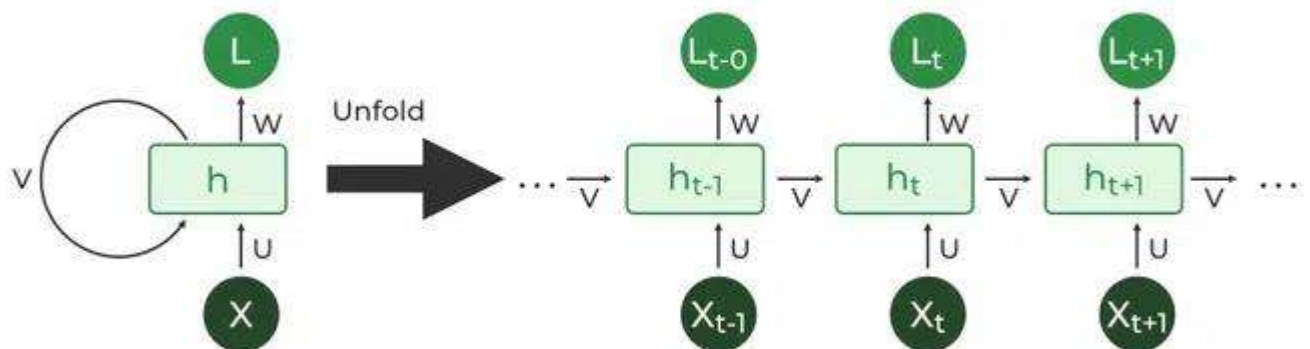
Key Features of RNN:

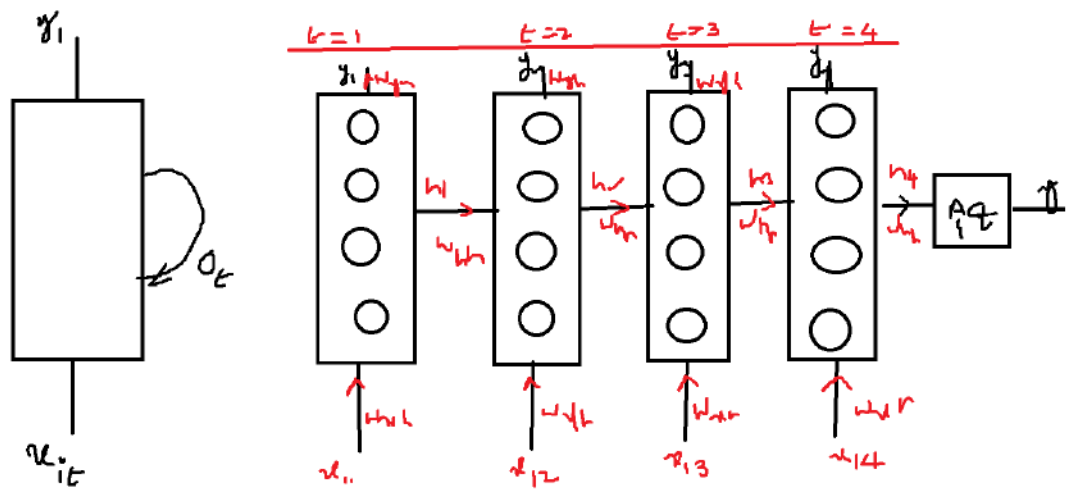
- **Sequential Processing:** Unlike feedforward neural networks, RNNs process data **one element at a time**, using their memory of past inputs.
- **Loops:** RNNs have loops within the network to allow information to persist.
- **Shared Weights:** The same weights are used at each time step, which helps in generalizing across time steps.

Applications:

- **Natural Language Processing (NLP):** text generation, sentiment analysis, language translation
- **Time Series Forecasting:** stock price prediction, weather prediction
- **Speech Recognition**
- **Music Generation**

Basic Architecture:





time	Input(x)	Input weights	hidden weights	hidden layer output(h)	output weights	Output (y)
$t = 1$	x_1	w_{xh}	0	$h_1 = \tanh(x_1 * w_{xh} + b)$	w_{yh}	$y_1 = h_1 w_{yh} + b_y$
$t = 2$	x_2		w_{hh}	$h_2 = \tanh(x_2 * w_{xh} + h_1 * w_{hh} + b)$		
$t = 3$	x_3					
$t = 4$	x_4					

<i>time</i>	<i>Input(x)</i>	<i>Input weights</i>	<i>hidden weights</i>	<i>hidden layer output(h)</i>	<i>output weights</i>	<i>Output (y)</i>
$t = 1$	x_1	w_{xh}	0	$h_1 = \tanh(x_1 * w_{xh} + b)$	w_{yh}	$y_1 = \tanh(h_1 w_{yh} + b_y)$
$t = 2$	x_2	w_{xh}	w_{hh}	$h_2 = \tanh(x_2 * w_{xh} + h_1 * w_{hh} + b)$	w_{yh}	$y_2 = \tanh(h_2 w_{yh} + b_y)$
$t = 3$	x_3	w_{xh}	w_{hh}	$h_3 = \tanh(x_3 * w_{xh} + h_2 * w_{hh} + b)$	w_{yh}	$y_3 = \tanh(h_3 w_{yh} + b_y)$
$t = 4$	x_4	w_{xh}	w_{hh}	$h_4 = \tanh(x_4 * w_{xh} + h_3 * w_{hh} + b)$	w_{yh}	$y_4 = \tanh(h_4 w_{yh} + b_y)$

How to read this:

- **At time step 1:**
 - Input x_1 is passed into the RNN.
 - The hidden state h_1 is computed using the formula: $h_1 = \tanh(W_{xh} * x_1 + b)$.
 - Output y_1 is calculated based on h_1 : $y_1 = W_{yh} * h_1 + b_y$.
- **At time step 2:**
 - Input x_2 is passed in, and the hidden state h_2 is updated using the hidden state h_1 from the previous time step: $h_2 = \tanh(W_{xh} * x_2 + W_{hh} * h_1 + b)$.
 - Output y_2 is computed from h_2 .
- **And so on for the subsequent time steps.**

Step 1: Forward Pass

In the forward pass, we calculate the hidden states (h_t) and outputs (y_t) for each time step using the formula:

$$h_t = \tanh(x_t * w_{xh} + h_{t-1} * w_{hh} + b)$$

The output is then calculated as:

$$y_t = h_t * w_{yh} + b_y$$

At the end of the forward pass, we have the predicted outputs y_t for each time step and the corresponding hidden states h_t .

Step 2: Loss Calculation

The next step is to calculate the loss (error) between the predicted outputs y_t and the true labels y_t^{true} for each time step. A common loss function for sequence tasks is **Mean Squared Error (MSE)**, or **Cross-Entropy Loss** for classification tasks.

$$Loss = \sum (y_a, y_p)$$

Where L is the total loss for the entire sequence, and T is the number of time steps.

Step 3: Back Propagation

Chain Rule:

$$\text{loss w.r.t } y_p = \frac{\partial L}{\partial y_p}$$

update the weights at time $t = 4$

$$\frac{\partial L}{\partial w_{hh}}$$

those weights w_{hh} directly depends on y_p

$$\frac{\partial L}{\partial w_{hh}} = \frac{\partial L}{\partial y_p} * \frac{\partial y_p}{\partial w_{hh}}$$

$$w_{hh_{new}} = w_{hh} - \text{learning rate} * \frac{\partial L}{\partial w_{hh}}$$

$$\frac{\partial L}{\partial w_{xh}}:$$

h_4 depends on w_{xh}

y_p depends on h_4

Loss depends on y_p

$$\frac{\partial L}{\partial w_{xh}} = \frac{\partial L}{\partial y_p} * \frac{\partial y_p}{\partial h_4} * \frac{\partial h_4}{\partial w_{xh}}$$

Vanish Gradient problem

When training an RNN, gradients are propagated backward through each time step (from T to 1). But due to repeated multiplication by weight matrices and non-linear activation derivatives (like tanh or sigmoid), the gradients can shrink exponentially as they go further back in time.

Result: Earlier layers (i.e., earlier time steps) receive very small gradients, so their weights learn very slowly or not at all. This is called vanishing gradients.

Let's map it step-by-step:

Roadmap Step	Relevance to Seq2Seq Transformers
✓ Learn RNNs, LSTMs	Background for sequence modeling
✓ Understand Transformer architecture	Core to Seq2Seq models (encoder-decoder)
✓ Learn Self-Attention	Power behind encoder + decoder
✓ Learn Cross-Attention	Decoder uses this to attend to encoder
✓ Study models like T5, BART	These <i>are</i> sequence-to-sequence transformers
✓ Prompt engineering	Used with encoder-decoder models like T5
✓ Fine-tuning (LoRA, PEFT)	Applied on seq2seq models for specific tasks
✓ Use HuggingFace transformers	Hands-on with T5, BART, etc.

